4.

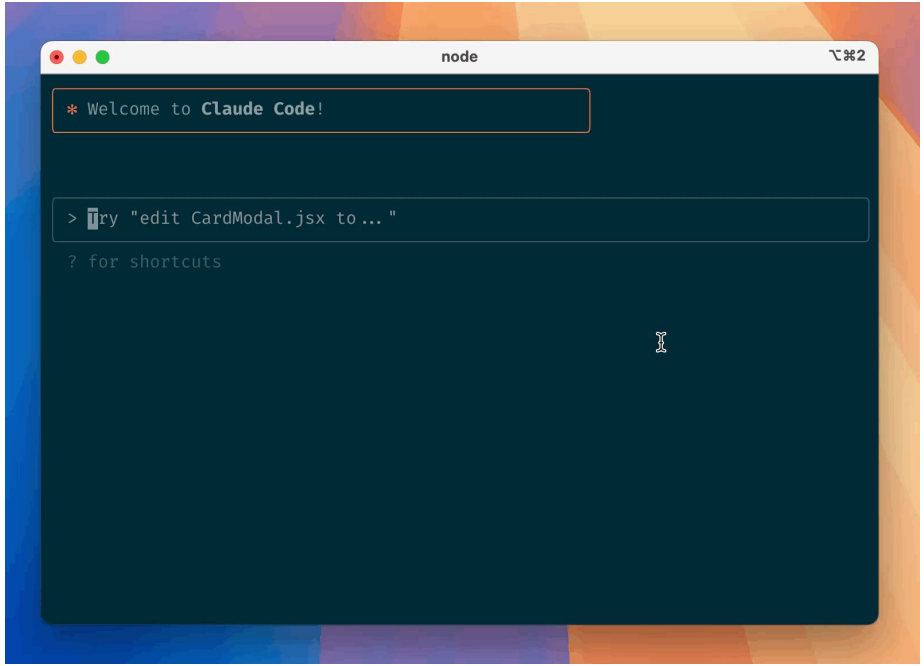
Beyond the Prompt: Building Autonomous LLM Agents

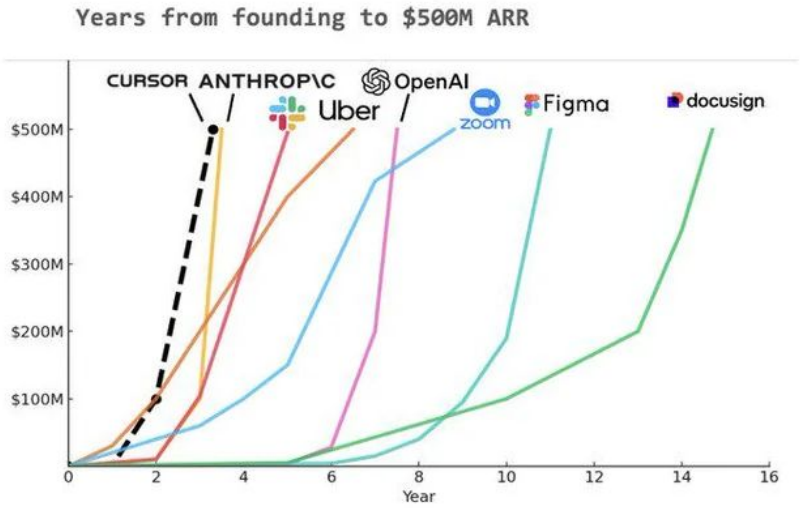
LLM Agent Business Value

> The Rise Of The **One-Person Unicorn**: How AI Agents Are Redefining Entrepreneurship **-** Forbes - Mars 2025

Y Combinator, 2024, October, Why Vertical LLM Agents Are The New $1 Billion SaaS Opportunities

Claude Code

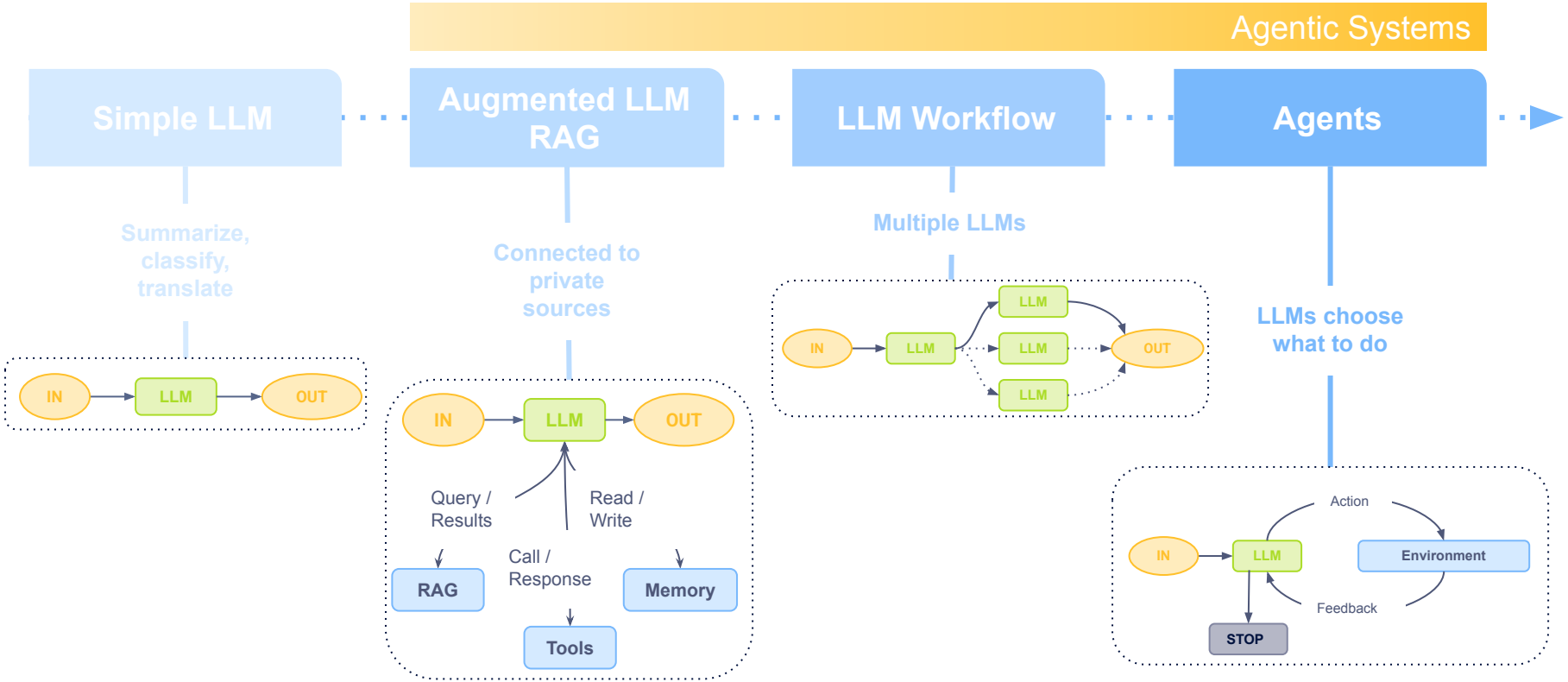




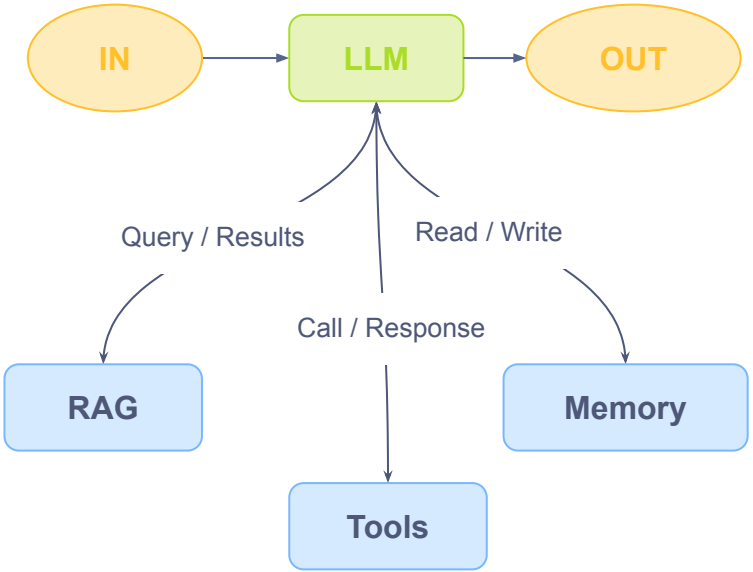
Claude Code

A profitable company

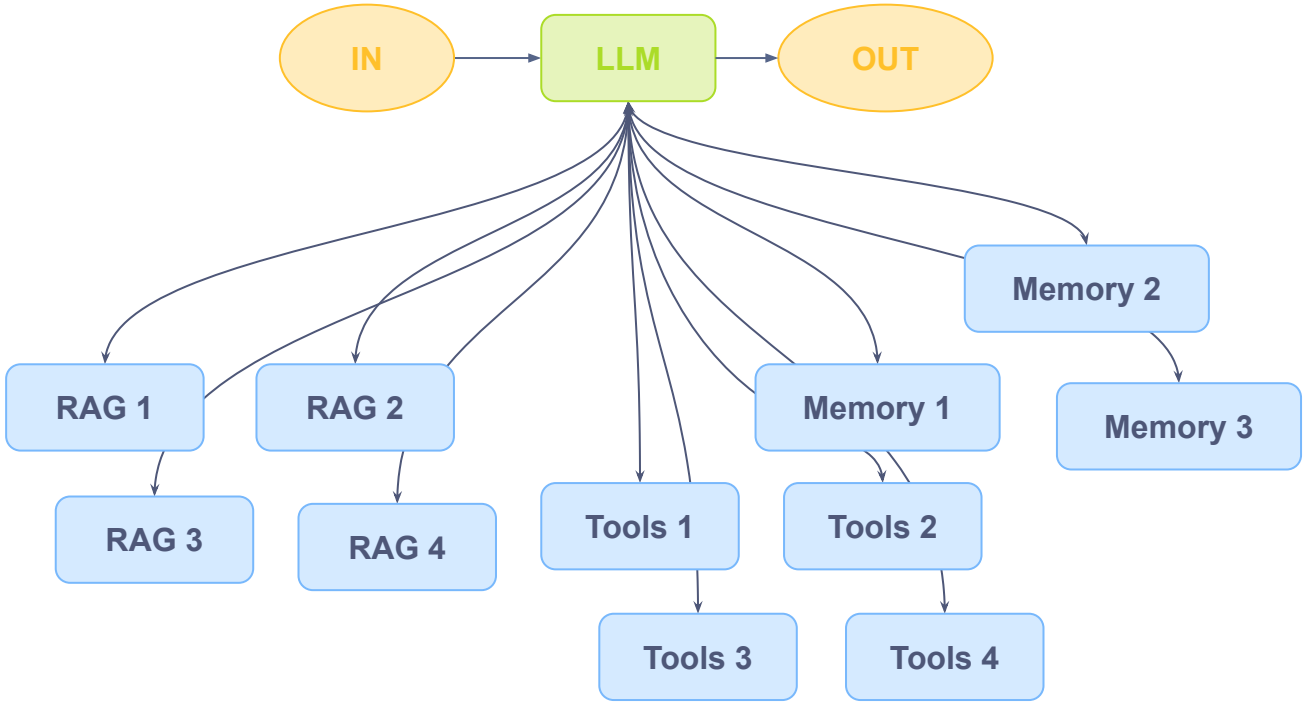
LLM Agents


Agentic Systems
Simple LLM
Augmented LLM RAG
LLM Workflow
Agents
Summarize, classify, translate
IN
LLM
OUT
Connected to private sources
IN
LLM
OUT
Query / Results
Read / Write
Call / Response
RAG
Memory
Tools
Multiple LLMs
IN
LLM
LLM
LLM
LLM
OUT
LLMs choose what to do
Action
IN
LLM
Environment
Feedback
STOP

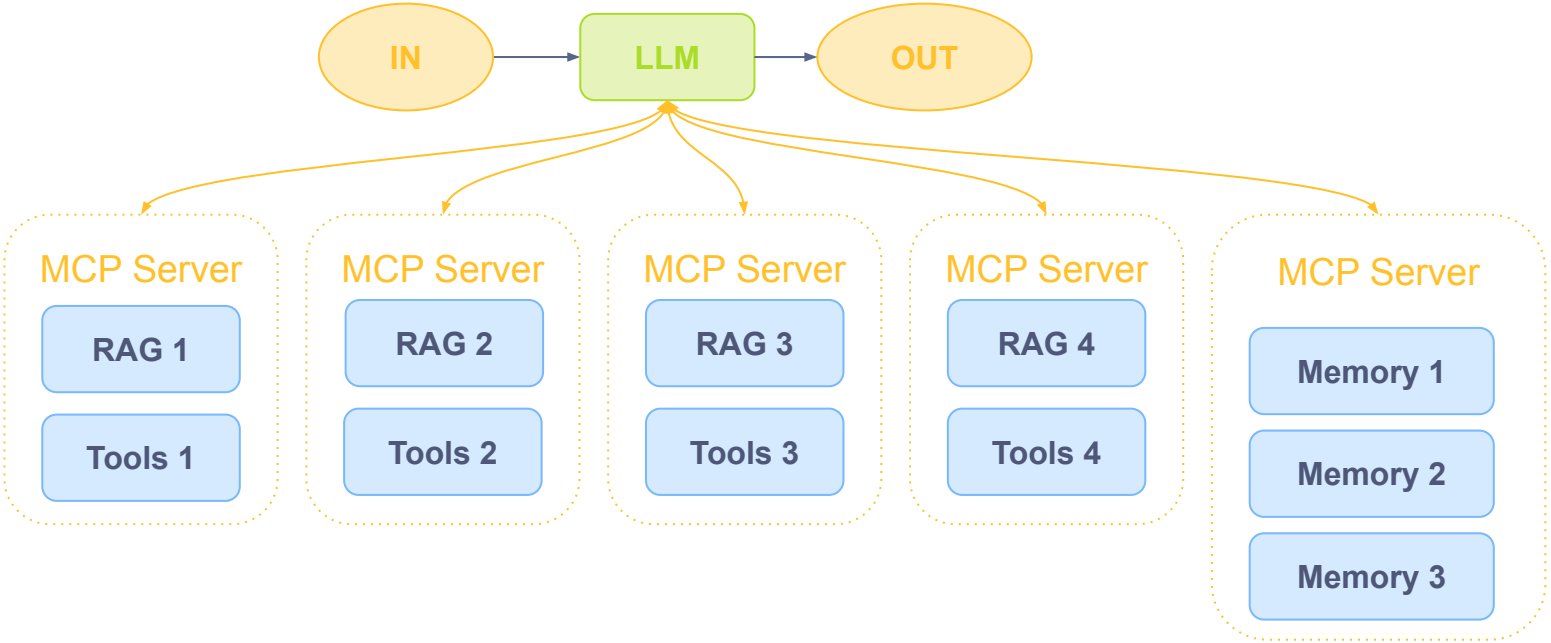
Simple to complex pipeline



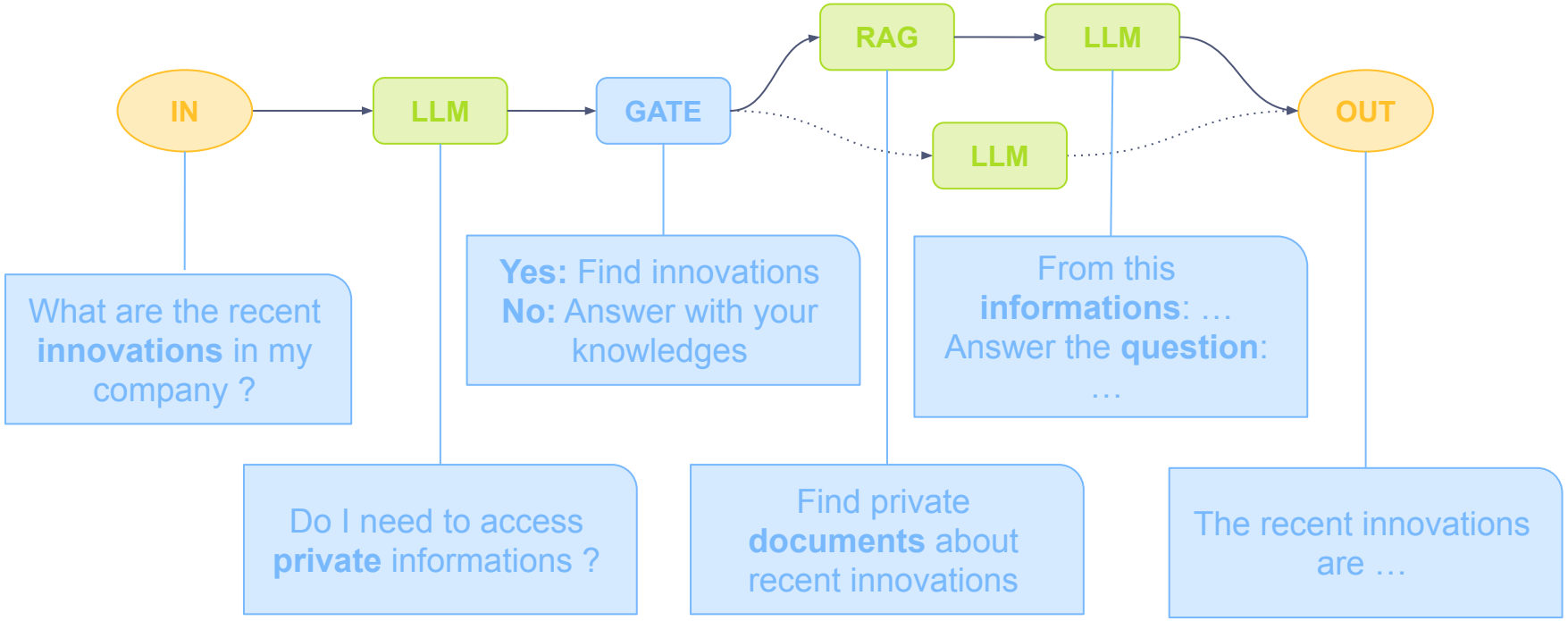
How to reduce complexity ?



How to reduce complexity ?



Orchestration Workflows

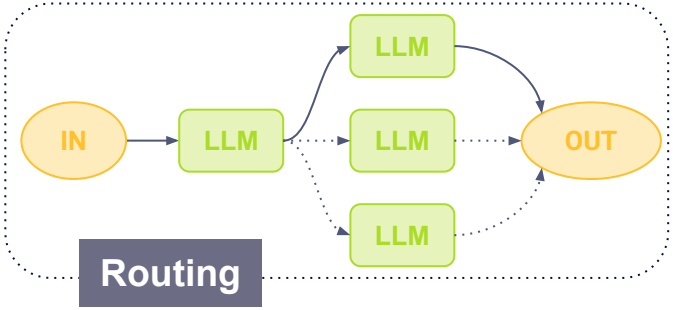


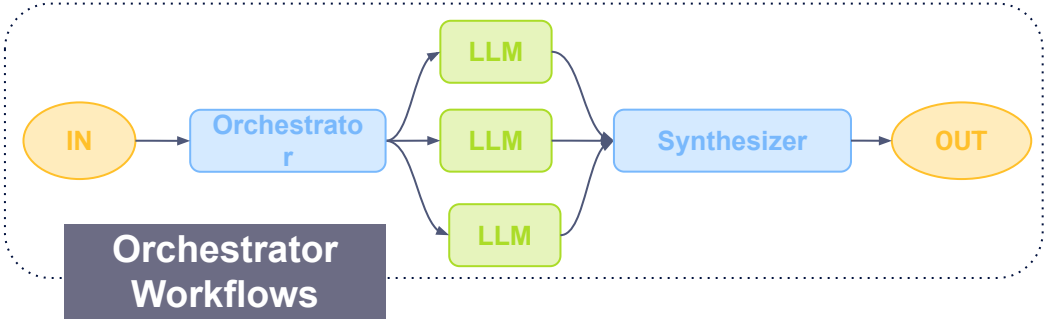
Orchestration Workflows

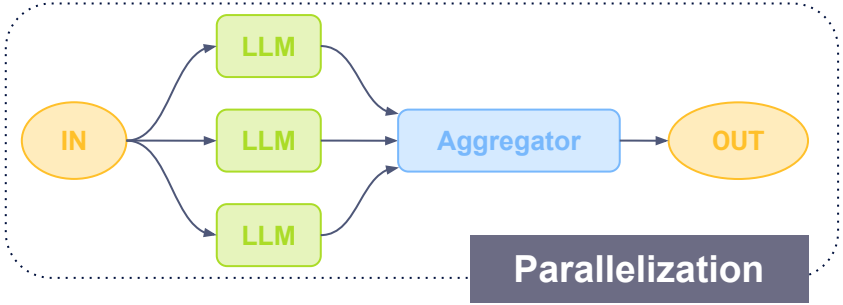
Anthropic - Building effective agents

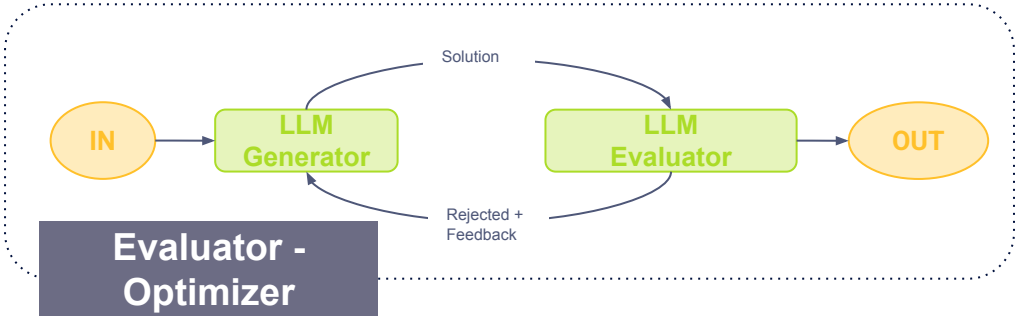
> We've worked with dozens of teams **building LLM agents** across industries.
> Consistently, the most **successful implementations use simple**, composable patterns **rather than complex** frameworks.

Orchestration Workflows

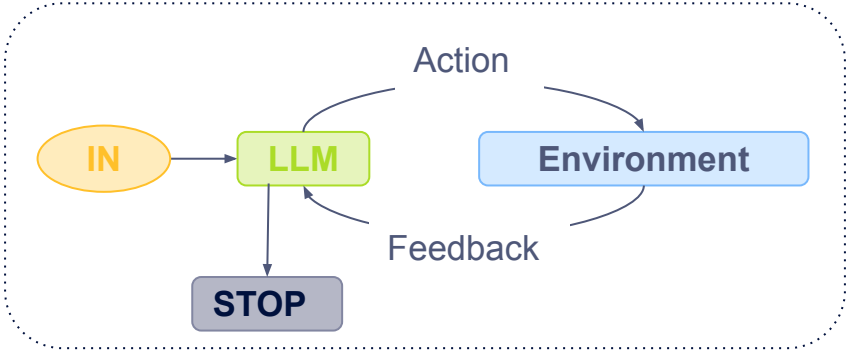






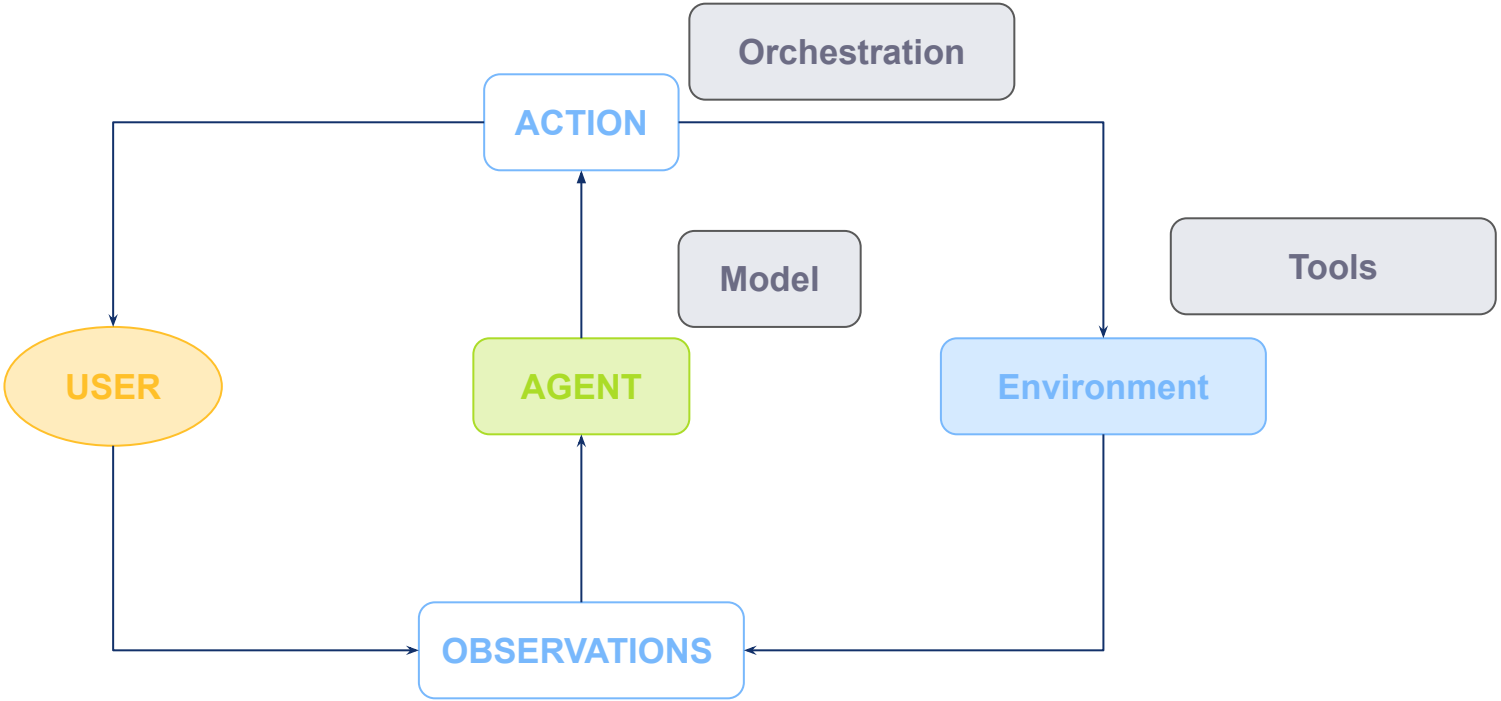


AI Agent



An **AI agent** is an autonomous system driven by a LLM capable of reasoning, planning and interacting with and environment in order to reach a goal.

AI Agent Components



LLM Agents Model

- **Reasoning, planning**: Being able to make the right choice
- **Interact with the environment (tools)**: MCP or function calling
- **Context Size**: Big enough to handle memory
- **Cost**

Rank	Model	Type	Vendor	Cost (I/O)	Avg Category Score (TSQ)
1st	claude-3-7-sonnet-20250219	Private	Anthropic	$3.00/$15.00	0.953
2nd	gemini-2.5-pro-preview-03-25	Private	Google	$1.25/$10.00	0.941
3rd	gemini-2.0-flash-001	Private	Google	$0.15/$0.60	0.938
4	gemini-2.0-flash-lite-001	Private	Google	$0.07/$0.30	0.933
5	gpt-4.1-2025-04-14	Private	OpenAI	$2.00/$8.00	0.918
6	mistral-small-2503	Open source	Mistral	$0.10/$0.30	0.912
7	gpt-4.1-mini-2025-04-14	Private	OpenAI	$0.40/$1.60	0.910
8	deepseek-v3-0324	Open source	Deepseek	$0.27/$1.10	0.905
9	gpt-4o-2024-11-20	Private	OpenAI	$2.50/$10.00	0.900
10	gpt-4.5-preview-2025-02-27	Private	OpenAI	$75.00/$150.00	0.900
11	gemini-1.5-flash	Private	Google	$0.07/$0.30	0.895
12	o4-mini-2025-04-16	Private	OpenAI	$1.10/$4.40	0.889

http://huggingface.co/spaces/galileo-ai/agent-leaderboard

Tools

User

What is the weather in Paris today ?

Assistant

I'm sorry, I cannot provide current weather information as I do not have live data access. I recommend checking a reliable weather website or using a weather app for the most up-to-date information on the weather in Paris.

Good Bad

GPT 3.5 Turbo

What is the weather in Paris today ?

Show thinking

Currently

21°C

Partly cloudy

High: 26° Low: 15° Precip: 5%

Monday · Paris, France

12:00	13:00	14:00	15:00	16:00	17:00	18:00	19:00	20:00
21°	22°	23°	24°	25°	26°	26°	26°	25°
0%	0%	0%	0%	5%	5%	5%	5%	5%

Google Weather

Gemini 2.5 Flash

Tools

A **tool** is a **function or API** that a language model can call during reasoning to perform tasks beyond its built-in capabilities.

Tools allow LLMs to:

- Access **real-time** or **external data**
- Perform **actions** (e.g., send an email, query a database)
- Call business logic or automation functions
- **Retrieve** and process **documents** or knowledge
- Interact with external systems (APIs, databases, web services, etc.)

Tools

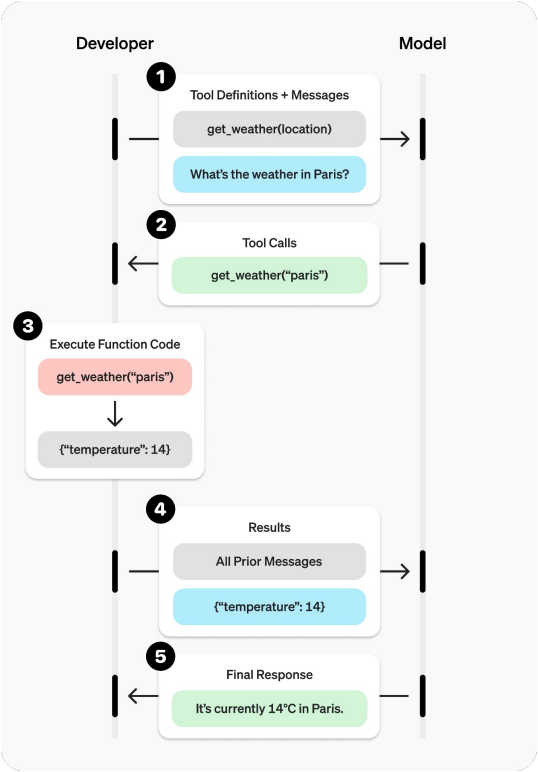
```json
{
  "name": "get_weather",
  "description": "Get the current weather information for a given city.",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "The name of the city to get the weather for"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "The temperature unit"
      }
    },
    "required": ["city"]
  }
}
```

How the LLM see the tool ?
For OpenAI Function Calling or LangChain Tool

```python
import requests

def get_weather(city: str, unit: str = "celsius") -> dict:
    """Calls a weather API and returns current weather for a given city."""

    # For demo, replace with a real weather API like OpenWeatherMap
    api_key = "YOUR_API_KEY"
    units = "metric" if unit == "celsius" else "imperial"

    response = requests.get(
        f"https://api.openweathermap.org/data/2.5/weather",
        params={"q": city, "units": units, "appid": api_key}
    )

    if response.status_code != 200:
        return {"error": f"Failed to get weather for {city}"}

    data = response.json()

    return {
        "city": data["name"],
        "temperature": data["main"]["temp"],
        "description": data["weather"][0]["description"],
        "humidity": data["main"]["humidity"],
        "unit": unit
    }
```

How we get the weather ?

Tools



Function calling workflow

Tool best practises

- Write clear and detailed function names, parameter descriptions, and instructions.
- Explicitly **describe the purpose** of the function and each parameter (and its format), and what the output represents.
- **Use the system prompt** to describe when (and when not) to use each function. Generally, tell the model exactly what to do.
- **Include examples** and edge cases, especially to rectify any recurring failures. (Note: Adding examples may hurt performance for reasoning models.)
- **Combine functions** that are always called in sequence. For example, if you always call mark_location() after query_location(), just move the marking logic into the query function call.
- Keep the **number of functions small** for higher accuracy.

Open AI good practises

Under the hood

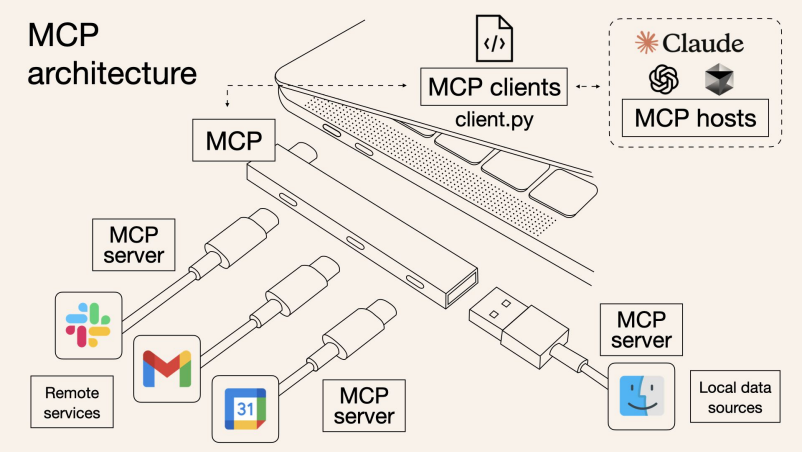
```
[
  {
    "role": "user",
    "content": "What's the weather like in Tokyo right now?"
  },
  {
    "role": "assistant",
    "function_call": {
      "name": "get_weather",
      "arguments": "{ \"city\": \"Tokyo\", \"unit\": \"celsius\" }"
    }
  },
  {
    "role": "function",
    "name": "get_weather",
    "content": "{ \"city\": \"Tokyo\", \"temperature\": 28.3, \"description\": \"sunny\",
  },
  {
    "role": "assistant",
    "content": "The current weather in Tokyo is 28.3°C and sunny."
  }
]
```

MCP

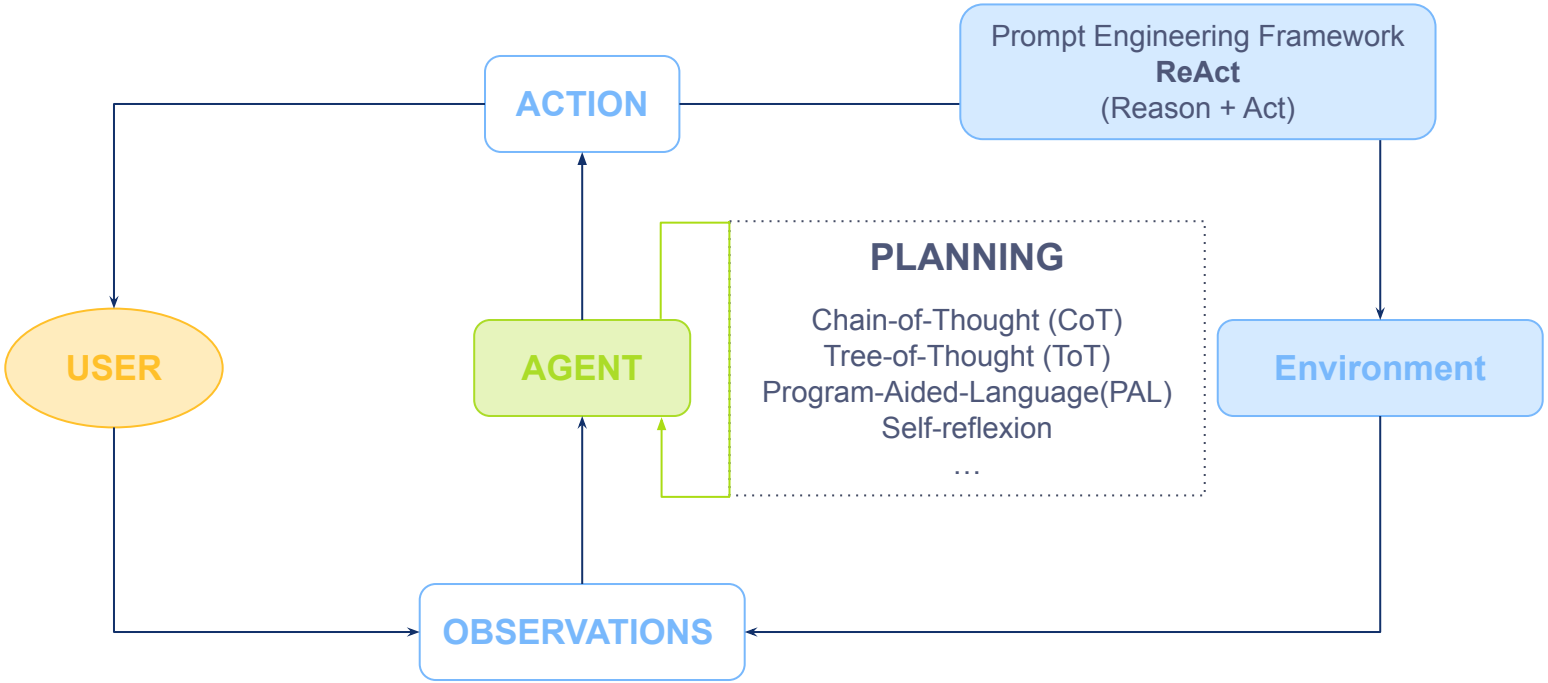
MCP (Model Context Protocole) is an open protocol that standardizes how applications provide context to LLMs. Think of MCP like a USB-C port for AI applications.

Just as USB-C provides a standardized way to connect your devices to various peripherals and accessories, MCP provides a standardized way to connect AI models to different data sources and tools.

The most popular MCP server: Smithery AI
Build your own MCP server: MCP: Get started (Python)



Orchestration



Orchestrator - Chain of Thought

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

PAL: Program-Aided Language

PAL add an intermediate reasoning step to transform a problem into Python code.

PAL distinguishes **reasoning** and **solving** in two separate steps

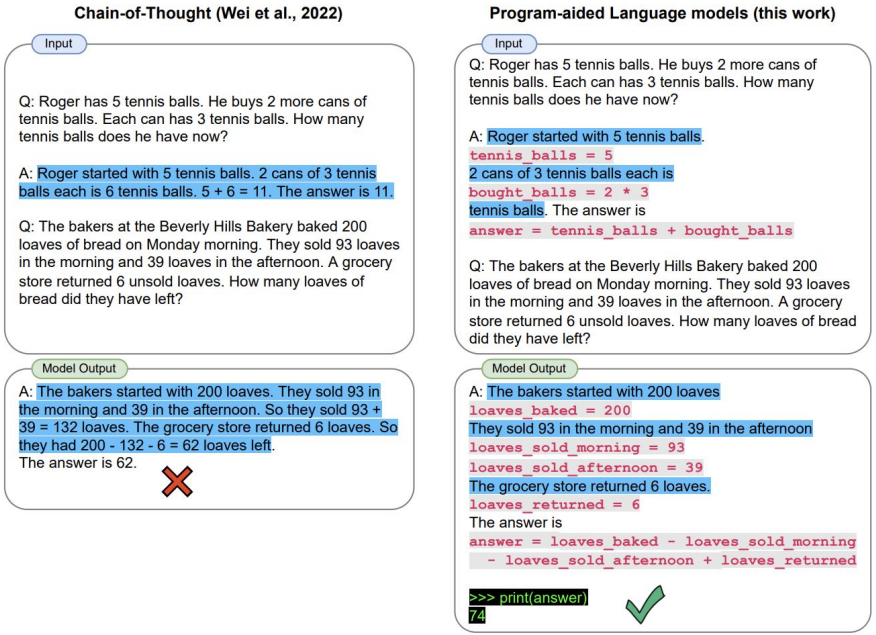
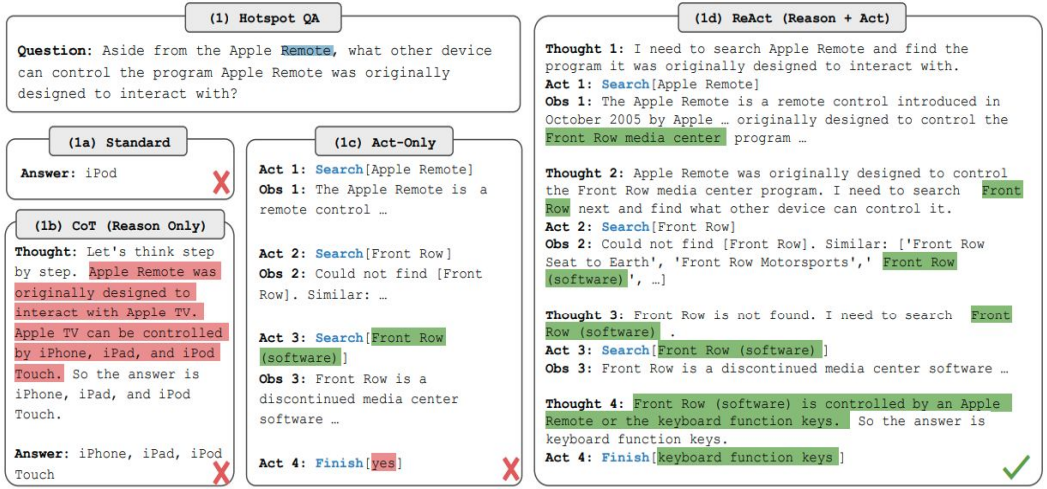


Figure 1: A diagram illustrating PAL: Given a mathematical reasoning question, Chain-of-thought (left) generates intermediate reasoning steps of free-form text. In contrast, Program-aided Language models (PAL, right) generate intermediate steps *and* Python code. This shifts the role of *running* the reasoning steps from the language model to the Python interpreter. The final answer is obtained by running the generated reasoning chain. Chain-of-thought reasoning is highlighted in blue; PAL steps are `highlighted in gray and pink`; the Python interpreter run is highlighted in black and green.

Gao & Al, 2022, PAL: Program-aided Language Models

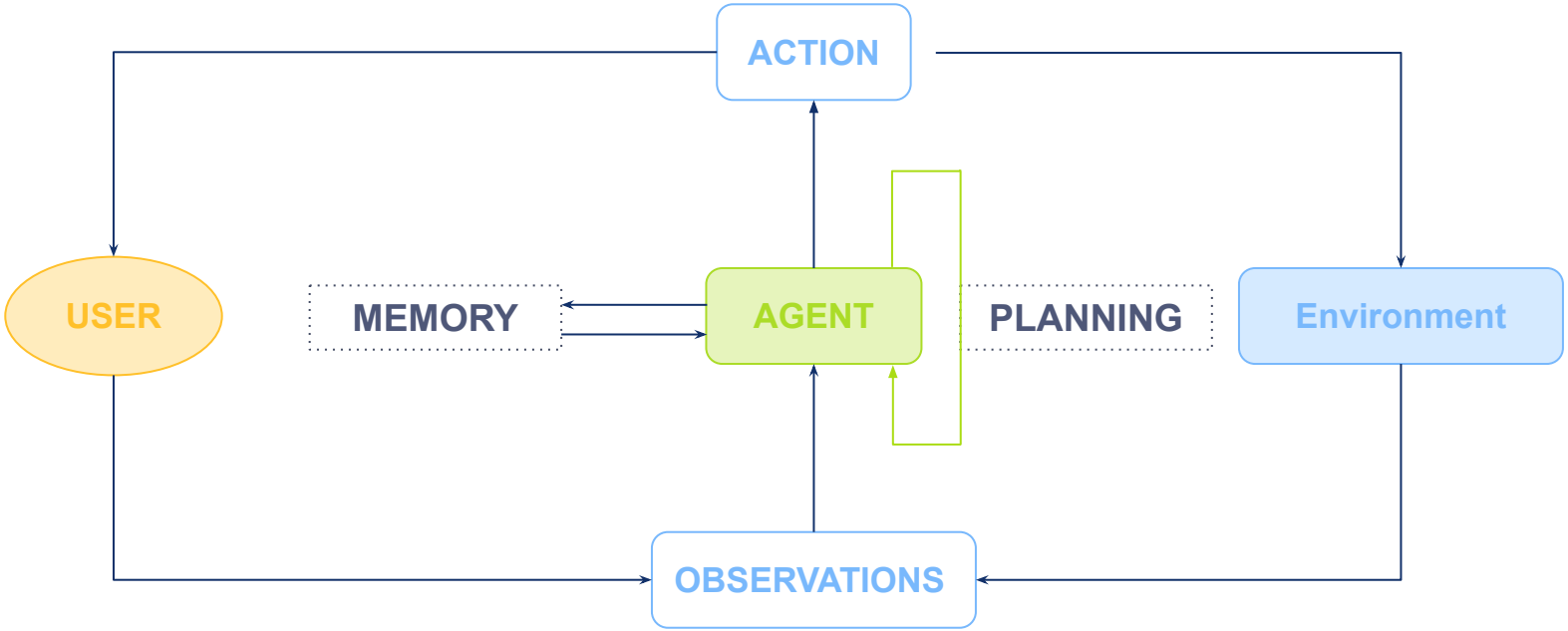
Synergizing Reasoning and Acting in Language Models (ReAct)

Prompt Method[a]	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	**64.6**
ReAct → CoT-SC	**35.1**	62.0
Supervised SoTA[b]	67.5	89.5

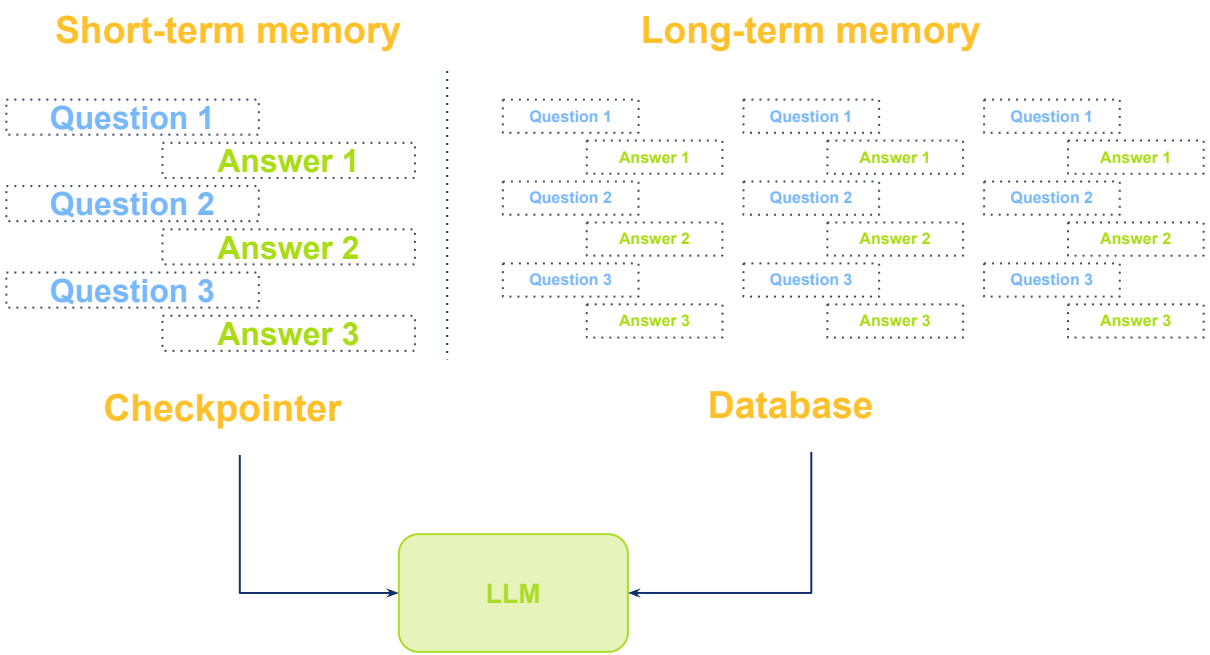


Yao & Al, 2023, Synergizing Reasoning and Acting in Language Models (ReAct)

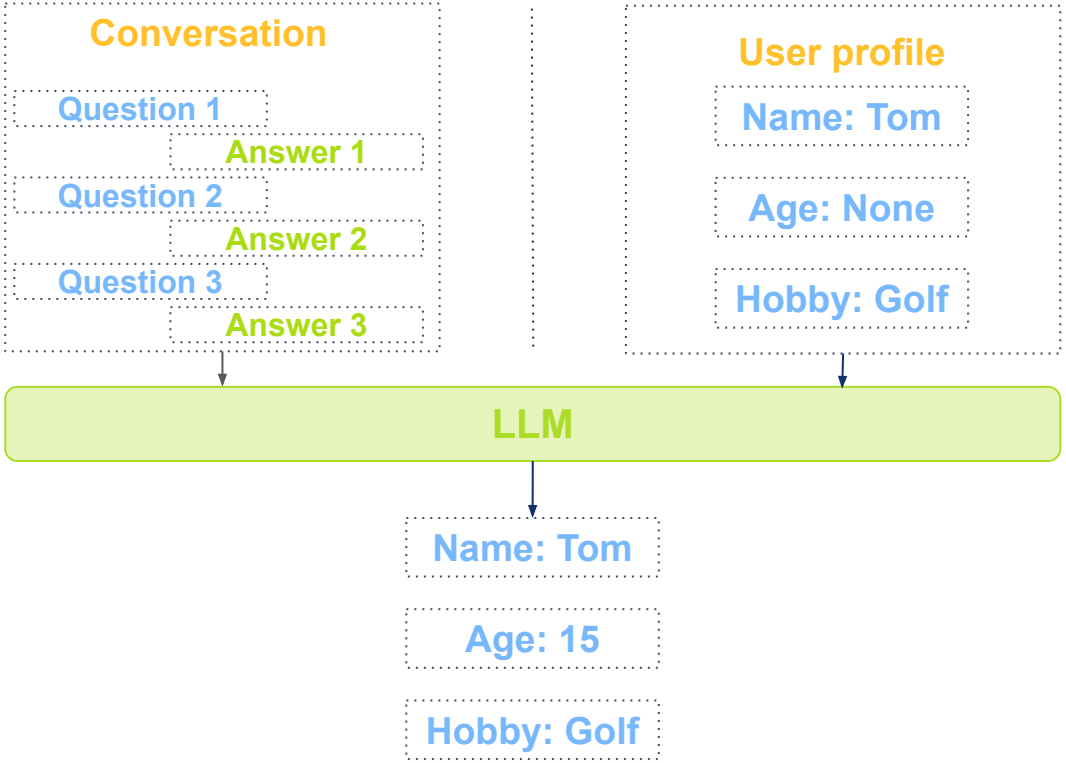
Memory


ACTION
USER
MEMORY
AGENT
PLANNING
Environment
OBSERVATIONS

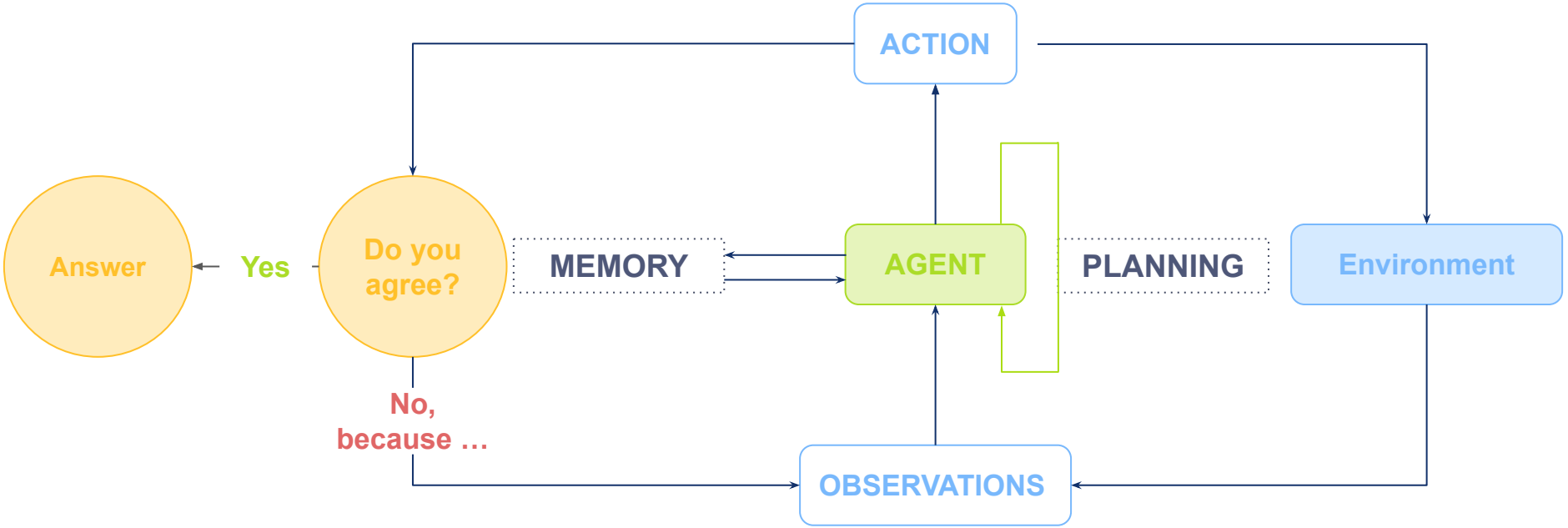
Memory


Short-term memory
Long-term memory
Question 1
Answer 1
Question 2
Answer 2
Question 3
Answer 3
Question 1
Answer 1
Question 2
Answer 2
Question 3
Answer 3
Question 1
Answer 1
Question 2
Answer 2
Question 3
Answer 3
Question 1
Answer 1
Question 2
Answer 2
Question 3
Answer 3
Checkpointer
Database
LLM

Memory


Conversation
Question 1
Answer 1
Question 2
Answer 2
Question 3
Answer 3
User profile
Name: Tom
Age: None
Hobby: Golf
LLM
Name: Tom
Age: 15
Hobby: Golf

Human in the loop

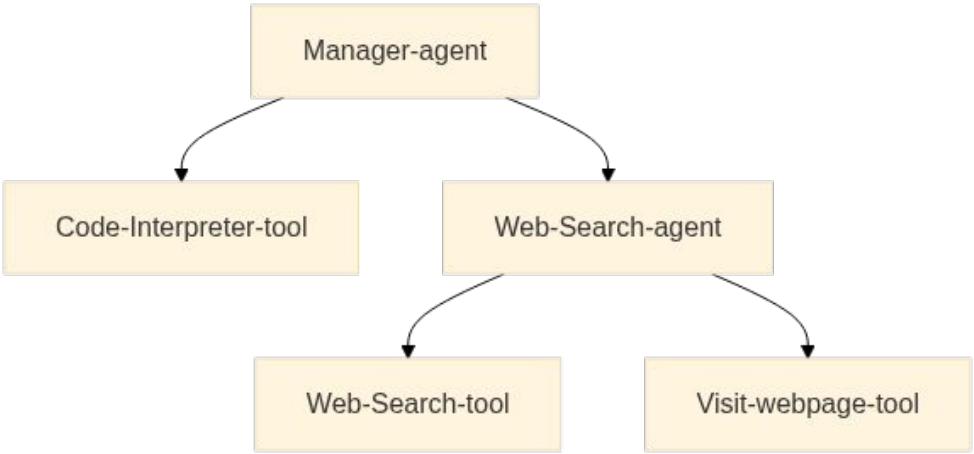


Multi Agent

Definition: Multi-agent systems enable specialized agents to collaborate on complex tasks, improving modularity, scalability, and robustness.

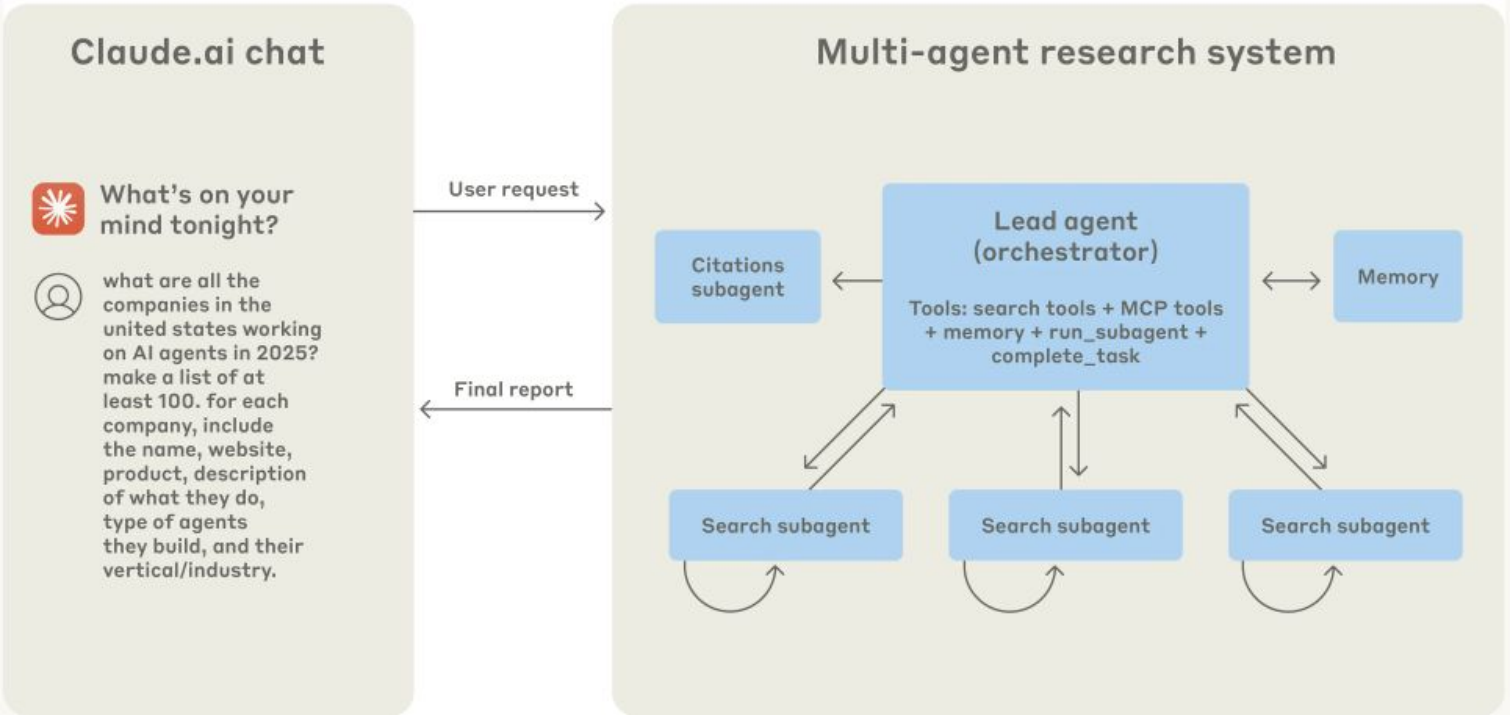
Instead of relying on a single agent, tasks are distributed among agents with distinct capabilities.

A **multi-agent system** consists of multiple specialized agents working together under the coordination of an Orchestrator Agent. This approach enables complex workflows by distributing tasks among agents with distinct roles.



Hugging Face, Multi-Agent Systems

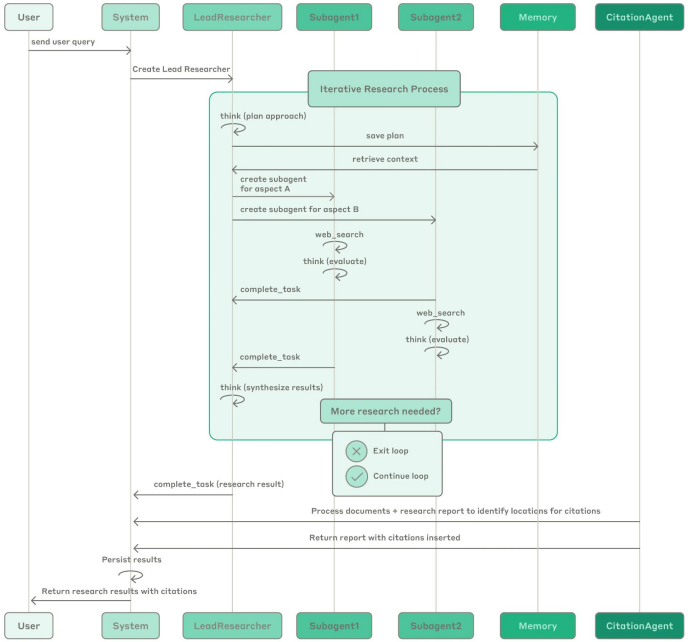
Multi Agent



Anthropic - How we built our multi-agent research system

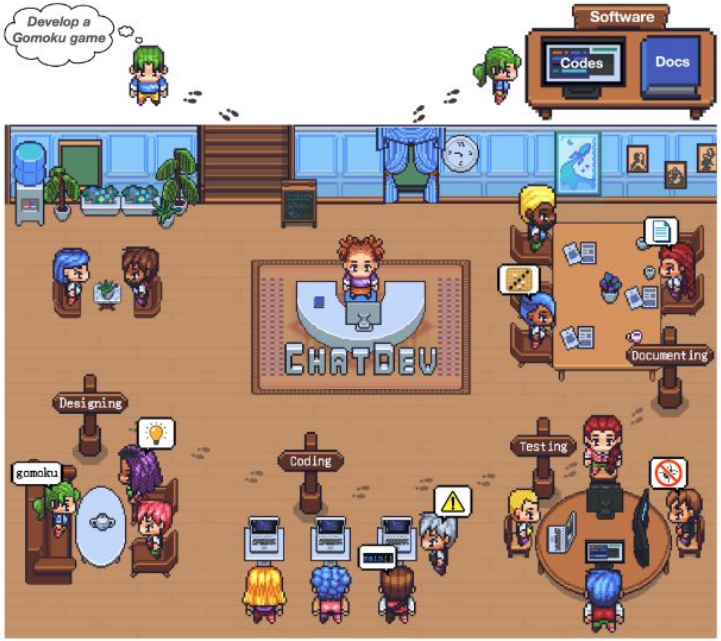
Multi Agent

Multi-agent System Process Diagram



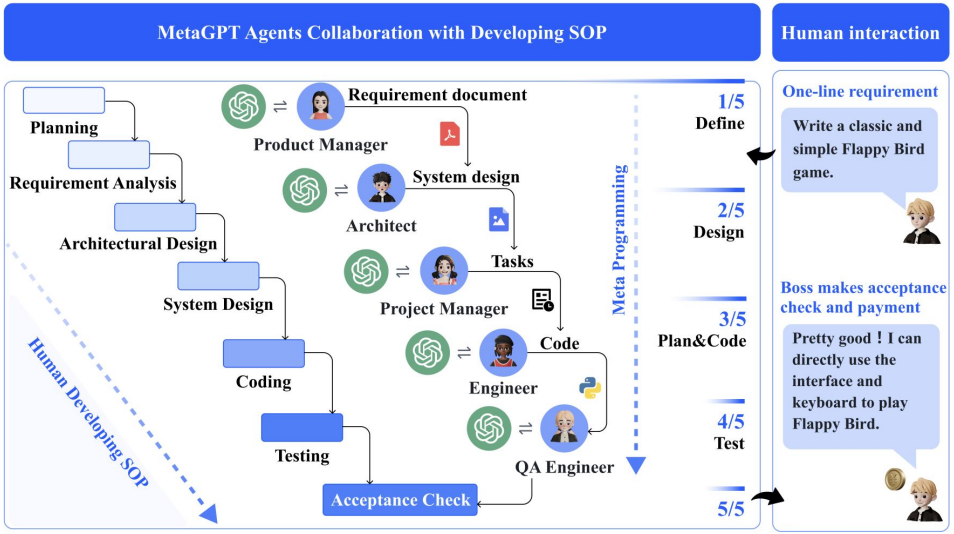
[Anthropic - How we built our multi-agent research system]

Multi Agent Example



ChatDev, 25K ⭐

Multi Agent Example



MetaGPT, 56K ⭐

References

Papers:
Gao & Al, 2022, PAL: Program-aided Language Models
Yao & Al, 2023, Synergizing Reasoning and Acting in Language Models (ReAct)

Blogs:
Tools - Open AI good practises
Anthropic - Building Effective Agents
Anthropic - How we built our multi-agent research system
Hugging Face - Multi-Agents System

Others:
Most Famous MCP - Smithery AI